JavaScript: Persistencia de datos en cliente

Alicia Vázquez
@aliciaFPInf

BY NC SA

Persistencia

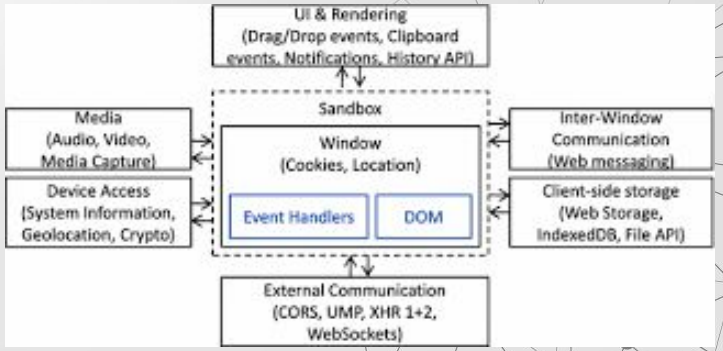
El protocolo **HTTP** es **stateless** (sin estado), es decir que se trata cada petición como una transacción independiente que no tiene relación con cualquier solicitud anterior, de modo que la comunicación se compone de pares independientes de solicitud y respuesta. **El servidor no mantiene memoria de nada de lo ocurrido anteriormente.**

Por otro lado, **Javascript** no tiene acceso directo ni al sistema de archivos del cliente ni del servidor. JS se ejecuta en un entorno llamado **sandbox** (entorno aislado).

Para poder guardar información en el cliente, los navegadores proporcionan librerías que forman parte de las últimas versiones de JS ES6 .

También veremos cómo el servidor puede leer cookies enviadas automáticamente por el navegador (útil para sesiones y tokens), y cómo conectar el almacenamiento del cliente con la API REST.

- Las principales son las **Cookies** y **LocalStorage**
  últimamente también **IndexedDB**
  pero no es soportado por todos los navegadores.



01

Cookies

enlace

Cookies

Las cookies son pequeños archivos de texto que se **almacenan en tu dispositivo** cuando visitas una página web. Estos archivos contienen información sobre tus interacciones con la página, y se utilizan para mejorar la experiencia de usuario y personalizar el contenido.

Son pequeños ficheros de texto en los que podemos guardar información. Es directamente el servidor quien pide al usuario permiso para poder almacenar información en local.

- Javascript tiene comandos específicos para manipular esos ficheros.
- Se guardan variables y su valor *(key, value)*. Máximo 4 KB por cookie, si nos pasamos se truncará.
- Las cookies sólo se pueden modificar desde el dominio que las ha creado. Máximo 20 cookies por dominio.
- Las cookies son las que, por ejemplo, mantienen una sesión (ej. Cesta compra) o ayudan a la publicidad contextual (ej. Hábitos de navegación).

Cookies: tipos

Hay varios tipos de cookies, como las **temporales** o de **sesión**, que se eliminan automáticamente cuando cierras tu navegador, y las **permanentes** o **persistentes**, que se almacenan en tu dispositivo por un período de tiempo determinado. Las cookies también pueden ser **propias**, creadas por la página web que visitas, o de terceros, creadas por otras empresas.

Las cookies tienen diferentes finalidades, como las **técnicas** o **necesarias**, que permiten el funcionamiento básico de la página web y no se pueden desactivar. También hay cookies de **análisis**, que recopilan información sobre la actividad del usuario en la página para mejorar la experiencia, y cookies **publicitarias**, que se utilizan para mostrar anuncios personalizados y relevantes.

Otro tipo de cookies son las de personalización, que almacenan tus **preferencias** y **configuraciones** en la página, y las de redes sociales, que permiten compartir contenido en redes sociales desde la página web.

Cookies: Configuración

Las páginas web tienen la obligación de mostrar una opción para configurar las cookies, donde se pueden desactivar algunos tipos de cookies.

Si se borran las cookies, se perderán los inicios de sesión y las preferencias, y se tendrá que volver a iniciar sesión y reconfigurar las cookies.

Si se desactivan las cookies de análisis, no se registrará la actividad del usuario en la web, y si se desactivan las cookies de preferencias, la información de configuración no se recordará. La privacidad aumentará al desactivar cookies, pero también se perderá la personalización en la experiencia de navegación.

Se recomienda encontrar un equilibrio entre la privacidad y la experiencia.

Cookies: Implementación

Cada cookie almacena los siguientes datos:

- **Nombre de la cookie** (obligatorio)
- **Valor** de la misma
- **Expires**: timestamp en que se borrará (si no pone nada se borra al salir del dominio, al cerrar el navegador)
- **Max-age**: en lugar de expires podemos indicar aquí los segundos que durará la cookie antes de expirar
- **Path**: ruta desde dónde es accesible (/: todo el dominio, /xxx: esa carpeta y subcarpetas). Si no se pone nada sólo lo será desde la carpeta actual
- **Domain**: dominio desde el que es accesible. Si no ponemos nada lo será desde este dominio y sus subdominios

Se puede acceder a las cookies desde **document.cookie** que es una cadena con las cookies de nuestras páginas. Para trabajar con ellas conviene que creemos funciones para guardar, leer o borrar cookies

Cookies: Implementación

Seguridad:

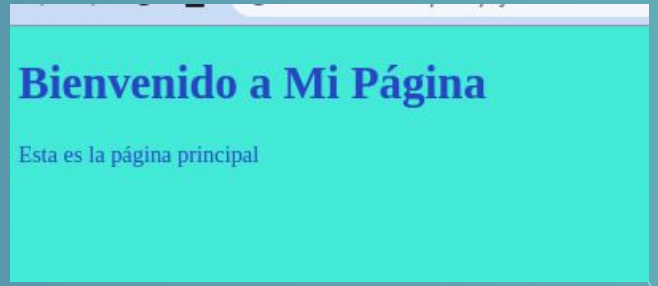
- **Secure**: si aparece indica que sólo se enviará esta cookie con HTTPS. En localhost no hace falta.
- **HttpOnly**: la cookie NO es accesible desde JavaScript (document.cookie).
  Solo viaja en cabeceras HTTP. Protege contra XSS.
  Solo puede crearla el servidor con el header Set-Cookie.
- **SameSite**: controla cuándo se envía la cookie en peticiones cross-site.
  SameSite=Strict ➔ solo en peticiones del mismo dominio
  SameSite=Lax ➔ valor por defecto moderno (navegadores)
  SameSite=None ➔ necesario para peticiones cross-origin (requiere Secure=true)

```
// El servidor (Express) crea una cookie HttpOnly así:
res.cookie('token', jwtValue, {
 httpOnly: true,    // ← JS no puede leerla
 secure: true,      // ← solo HTTPS
 sameSite: 'Lax',   // ← protección CSRF básica
 maxAge: 3600000    // ← 1 hora en ms
});
```

```javascript
// Guardar
function setCookie(nombre, valor, dias = 7) {
 const expires = new Date(Date.now() + dias * 864e5).toUTCString();
 document.cookie = `${nombre}=${encodeURIComponent(valor)}; expires=${expires}; path=/`;
}
// Leer  (patrón moderno con .find())
function getCookie(nombre) {
 return document.cookie
   .split('; ')
   .find(c => c.startsWith(nombre + '='))
   ?.split('=')[1]
   ?? null;
}
// Borrar  (expires en el pasado)
function deleteCookie(nombre) {
 document.cookie = `${nombre}=; expires=Thu, 01 Jan 1970 00:00:00 UTC; path=/`;
}
// Uso:
setCookie('usuario', 'Alicia', 7);
console.log(getCookie('usuario')); // → 'Alicia'
deleteCookie('usuario');
console.log(getCookie('usuario')); // → null
```

Ejercicio 1: Cookies

Utiliza alguna de las páginas que ya has hecho (la de inmobiliaria o la de videojuegos) y detectando si es la primera vez que entras en el dominio te abra una ventana de configuración, donde mediante un pequeño formulario elijas el color de fondo y el color de la fuente. Lo guardes en una cookie y luego lo recuperes cada vez que entres.



OJO, el navegador bloqueará **window.open()**. Para este ejercicio, debereis dar permiso en la barra de direcciones → permitir ques e abran pop-ups.

02

API Web Storage

Criteria	Local Storage	Session Storage	Cookies
Storage Capacity	5-10 mb	5-10 mb	4 kb
Auto Expiry	No	Yes	Yes
Server Side Accessibility	No	No	Yes
Data Transfer HTTP Request	No	No	Yes
Data Persistence	Till manually deleted	Till browser tab is closed	As per expiry TTL set

API Web Storage ¿qué es?

Hemos visto las cookies como mecanismo de almacenamiento de información en el lado del cliente, sin embargo existen maneras más actuales de hacerlo,

API WebStorage nos provee una forma de guardar información del lado del cliente en forma de **Key/Valor**, y a diferencia de las cookies esta información sólo es persistida del lado del cliente y no será enviada en cada petición que realicemos.

WebStorage nos brinda un conjunto de atributos y métodos para poder manipularlos. Existen dos implementaciones para ello:

- **LocalStorage**, nos permite almacenar información (strings) de manera permanente para múltiples ventanas o pestañas, de esta manera si cerramos el navegador no perdemos la información guardada a menos que decidamos eliminarla manualmente. Podemos almacenar entre 5 y 10 mb de información y a diferencia de las cookies, no será enviada al servidor.
- **SessionStorage**, es igual que el localStorage, con la salvedad que si cerramos el navegador/pestaña la información se pierde.

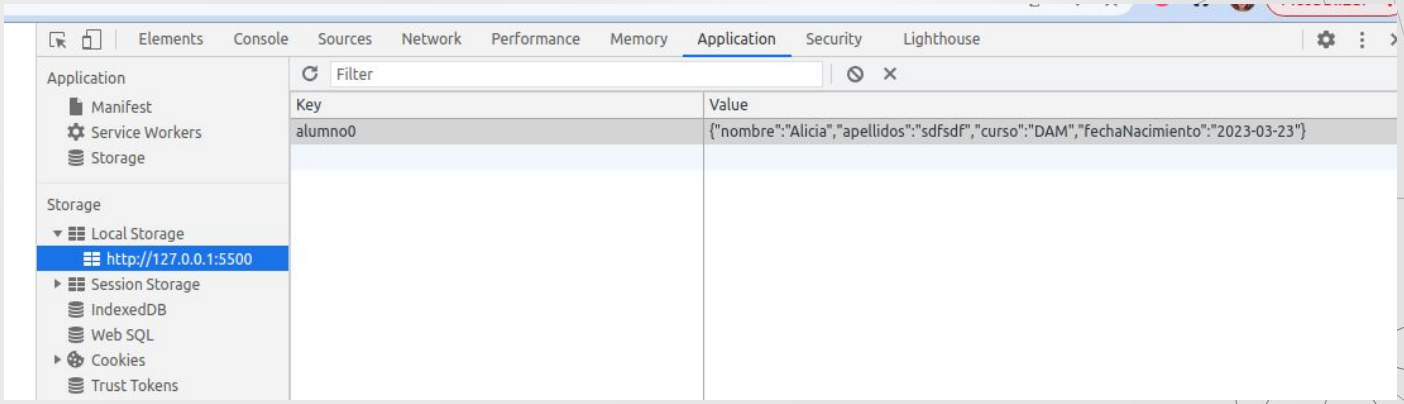
Nota: Si abrimos el devtools del navegador, en la pestaña "application" podemos ver la sección de Storage con todas las herramientas de persistencia que nos provee el navegador.

API Web Storage origen y limites

Al contrario que las cookies, los objetos de almacenaje web **no se envían al servidor** en cada petición. Debido a esto, podemos almacenar mucha más información. La mayoría de los navegadores modernos permiten almacenar, como máximo, **5 megabytes de datos** y tienen opciones para configurar estos límites.

Además, a diferencia de las cookies el servidor no puede manipular los objetos de almacenaje vía cabeceras HTTP, todo se hace vía JavaScript.

El almacenaje está vinculado al orígen (al triplete dominio/protocolo/puerto). Esto significa que distintos protocolos o subdominios tienen distintos objetos de almacenaje, no pueden acceder a otros datos que no sean los suyos.



API Web Storage métodos

Tanto el **localStorage** como el **sessionStorage** tienen los mismos métodos y propiedades.

- **.setItem(clave, valor)** – almacenar un par clave/valor.
- **.getItem(clave)** – obtener el valor por medio de la clave.
- **.removeItem(clave)** – eliminar la clave y su valor.
- **.clear()** – borrar todo.
- **.key(índice)** – obtener la clave de una posición dada.
- **.length** – el número de ítems almacenados.

localStorage

```
console.log(localStorage);
document.getElementById("addAlumno").addEventListener("click", addAlumno);
document.getElementById("delLocalStorage").addEventListener("click", delLocalStorage);

//Contador independiente para generar claves únicas (no usar localStorage.length: si borras y añades, genera colisiones)
let alumnoCounter = localStorage.length;

// Añadimos un nuevo alumno creando un objeto y serializándolo
function addAlumno(event) {
 const alumno = {
   nombre:          document.getElementById('nombre').value,
   apellidos:       document.getElementById('apellidos').value,
   curso:           document.getElementById('curso').value,
   fechaNacimiento: document.getElementById('fechaNacimiento').value
 };
 const key = 'alumno' + alumnoCounter++;          // clave única creciente
 localStorage.setItem(key, JSON.stringify(alumno)); // siempre stringify
}
// Eliminamos solo los items de alumnos (no tocamos otras claves del storage)
function delLocalStorage(event) {
 Object.keys(localStorage)
   .filter(key => key.startsWith('alumno'))
   .forEach(key => localStorage.removeItem(key));
 alumnoCounter = 0;                               // reiniciamos el contador
}
```

Problemas y trampas

1. **Guardar**.

Antes de guardar una cookie siempre hemos de usar **stringify** para convertirlo a texto.

```
localStorage.setItem('alumno', JSON.stringify({ nombre: 'Aina' }));
JSON.parse(localStorage.getItem('alumno')); // → { nombre: 'Aina' }
```

2. **getItem**

Si la clave no existente devuelve **null**, no <u>undefined</u>:

```
localStorage.setItem('alumno', JSON.stringify({ nombre: 'Aina' }));
JSON.parse(localStorage.getItem('alumno')); // → { nombre: 'Aina' }
```

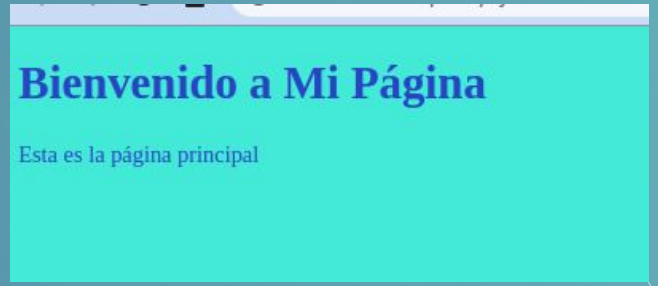
localStorage

```
// Comportameinto normal: Tanto si el usuario recarga, vuelve al inició o cierra y
vuelve → Empieza de cero.
sessionStorage.setItem('paso', '2');
sessionStorage.setItem('datosFormulario', JSON.stringify({
 nombre: 'Aina',
 email: 'aina@itb.cat'
}));
// Al cargar la página: recuperar el paso
const paso = sessionStorage.getItem('paso')  ?? '1';

// Diferencia clave con localStorage:
// - localStorage: persiste aunque cierres Chrome
// - sessionStorage: muere al cerrar la pestaña. (Una pestaña nueva del mismo dominio
tiene su propio sessionStorage)
```

Ejercicio 2: API WebStorage

Del ejercicio anterior, cambia las cookies por el localStorage.



Ejercicio 1-2

Conclusiones de los ejercicios anteriores

Característica	Con Cookies	Con LocalStorage
Sintaxis	document.cookie = "key=value; expires=..."	localStorage.setItem("key", "value")
Lectura	Requiere split(";") y trim()	localStorage.getItem("key") (Directo)
Capacidad	Muy limitada (4KB)	Amplia (~5MB)
Persistencia	Caduca cuando tú digas (expires)	No caduca nunca (hasta que se borre)

RECAP ¿Cuándo usar qué?

	Cookies	LocalStorage	SessionStorage
Capacidad	~4 KB	~5–10 MB	~5–10 MB
Expira	Configurable	Nunca (manual)	Al cerrar pestaña
Viaja al servidor	✅ Sí, siempre	❌ No	❌ No
Acceso JS	✅ Sí (salvo HttpOnly)	✅ Sí	✅ Sí
HttpOnly	✅ Sí (servidor)	❌ No	❌ No
Uso típico	Sesión, token JWT	Carrito, preferencias	Datos de un wizard/paso

Regla de oro:

- Si el dato necesita llegar al servidor → Cookie.
- Si es solo para el cliente → LocalStorage/SessionStorage.
- Secretos (contraseñas, tokens sensibles) → nunca en cliente.

03

CORS

Para conectar con nuestra API

CORS = Cross-Origin Resource Sharing

CORS es un mecanismo de seguridad que implementan los navegadores. Su función es actuar como un "portero" que decide si un sitio **web A** tiene permiso para pedirle datos a un **servidor B**.

Sin **CORS**, cualquier sitio web malicioso donde navegues podría intentar hacer peticiones en tu nombre a tu banco o a tu correo (siempre que tuvieras la sesión abierta), lo cual sería un caos.

La anatomía de un "Origen"

El navegador es extremadamente quisquilloso. Para él, dos URL son del mismo origen solo si coinciden en tres cosas: **Protocolo**, **Dominio** y **Puerto**.

Origen A	Origen B	¿Es el mismo origen?	Razón
http://localhost:3000	http://localhost:5500	No	El puerto es distinto.
http://miweb.com	https://miweb.com	No	El protocolo cambia (HTTP vs HTTPS).
http://api.com	http://app.com	No	El dominio es distinto.
http://localhost:3000	http://localhost:3000/api	Sí	El path no influye en el origen.

CORS ¿Cómo funciona el flujo?

Cuando haces un **fetch**() desde tu **front** (localhost:5500) hacia tu **back** (localhost:3000), ocurre lo siguiente:

1. **El Navegador añade una cabecera:** Envía la petición con un header llamado
   Origin: http://localhost:5500.

2. **El Servidor responde**: El servidor procesa la petición y, si está configurado para ello, devuelve una cabecera llamada Access-Control-Allow-Origin.

3. **El Navegador comprueba:**
   a. Si el servidor devolvió **Access-Control-Allow-Origin**: http://localhost:5500 (o un asterisco *), el navegador deja pasar los datos.
   b. Si esa cabecera no existe o no coincide, el navegador bloquea la respuesta y lanza el error que ves en consola.

```
const express = require('express');
const cors = require('cors');
const app = express();

// Opción A: Permitir todo (OJO)
app.use(cors());

// Opción B: Permitir solo tu front-end
app.use(cors({
 origin: 'http://localhost:5500'
}));
```

Nota importante: El servidor sí recibe la petición y sí envía los datos, pero es el navegador quien los "secuestra" y no permite que tu código JavaScript los vea por seguridad.

Desde el cliente

El puente entre **persistencia en cliente** y la **API**.

```
// GET — leer camisetas de la API (sustituye al
productosJSON)
async function cargarCamisetas() {
 try {
   const res = await
fetch('http://localhost:3000/api/camisetas');
   if (!res.ok) throw new Error(`Error ${res.status}`);
   const camisetas = await res.json();
   mostrarCamisetas(camisetas);
 } catch (err) {
   console.error('No se pudo conectar con la API:', err);
 }
}
```

```
// POST — enviar una comanda
async function enviarComanda(comanda) {
 const res = await fetch('http://localhost:3000/api/comandas', {
   method: 'POST',
   headers: { 'Content-Type': 'application/json' },
   body: JSON.stringify(comanda)
 });
 if (!res.ok) throw new Error(`Error ${res.status}`);
 return await res.json(); // devuelve el ticket
}
```

Desde el cliente

```
async function init() {
 // 1. Intentar recuperar del LocalStorage primero (respuesta inmediata)
 const cached = localStorage.getItem('camisetas');
 if (cached) mostrarCamisetas(JSON.parse(cached));

 // 2. Pedir siempre a la API para tener datos frescos
 const camisetas = await cargarCamisetas();

 // 3. Actualizar caché
 localStorage.setItem('camisetas', JSON.stringify(camisetas));
}
```

Ejercicio 3

Del ejericio de Alumnos y notas hecho en clase, conectate a la API (enciende el servidor node) y trata de acceder a los alumnos y las notas que tienes en el servidor desde una pequeña página web que muerte los datos recibidos.